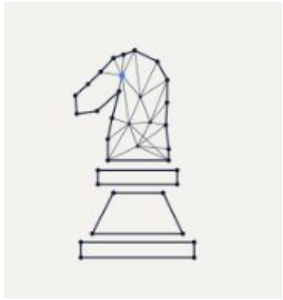
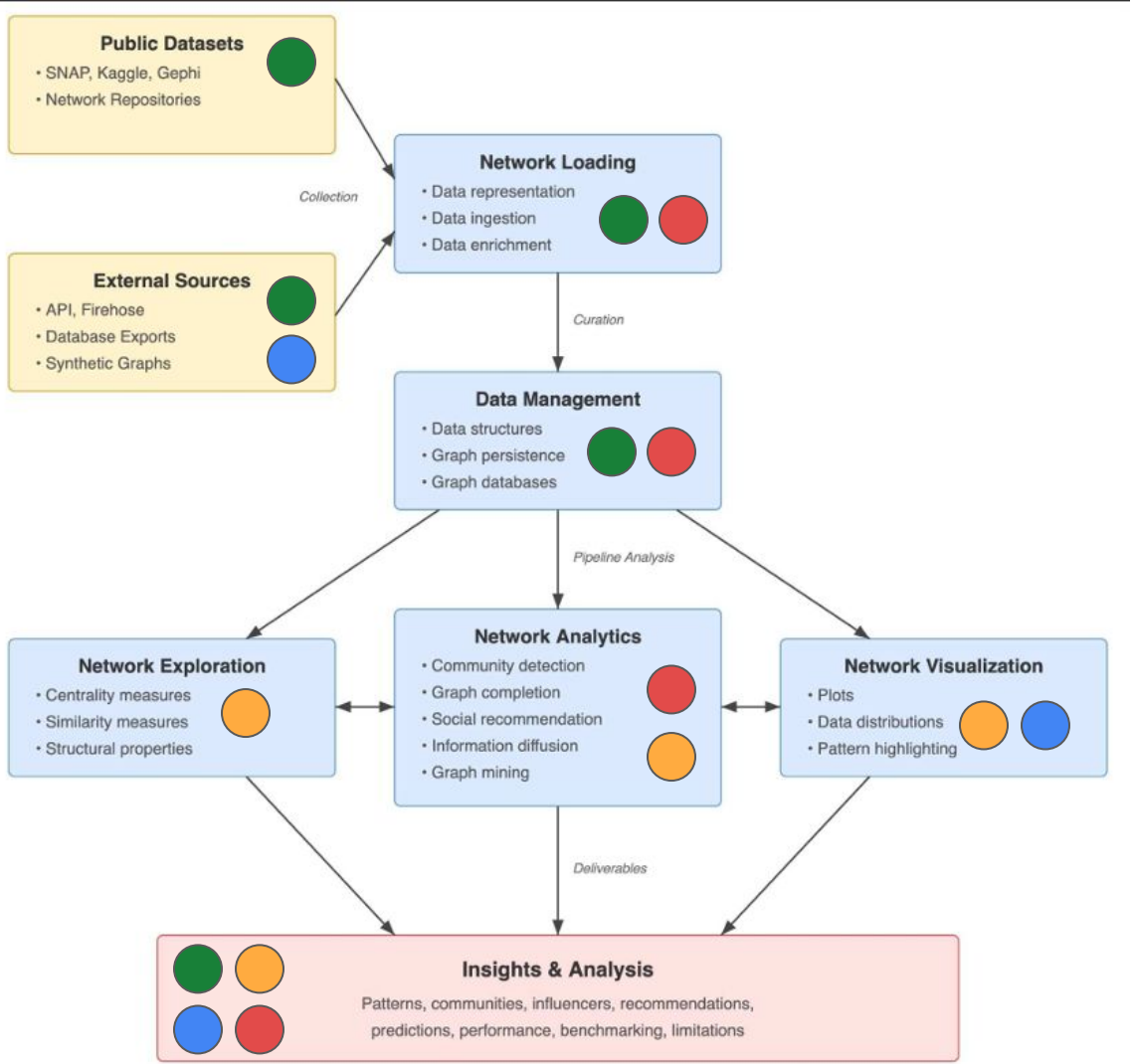
A Graph-Based Approach to Optimal Path Search Using Probability Heuristics

Béla Romeo Bernasconi
Andrés Santiago Martínez Hernández
Sébastien Pierret
Ali Abdel Ghaffar



Contributions

- Bela
- Sebastien
- Ali
- Santiago

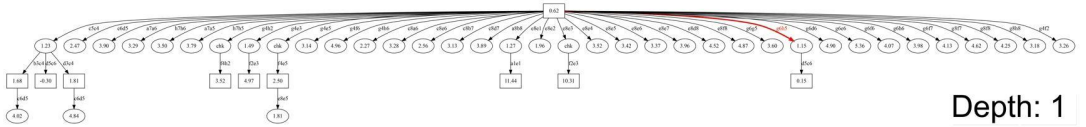


Computational chess

Shannons number:
10^{120}

Number of plies (half-moves)	Number of possible games[2]	Number of possible positions[3]
1	20	20
2	400	400
3	8,902	5362
4	197,281	72,078
5	4,865,609	822,518
6	119,060,324	9,417,681
7	3,195,901,860	96,400,068
8	84,998,978,956	988,187,354
9	2,439,530,234,167	9,183,421,888
10	69,352,859,712,417	85,375,278,064
11	2,097,651,003,696,806	726,155,461,002
12	62,854,969,236,701,747	
13	1,981,066,775,000,396,239	
14	61,885,021,521,585,529,237	
15	2,015,099,950,053,364,471,960	

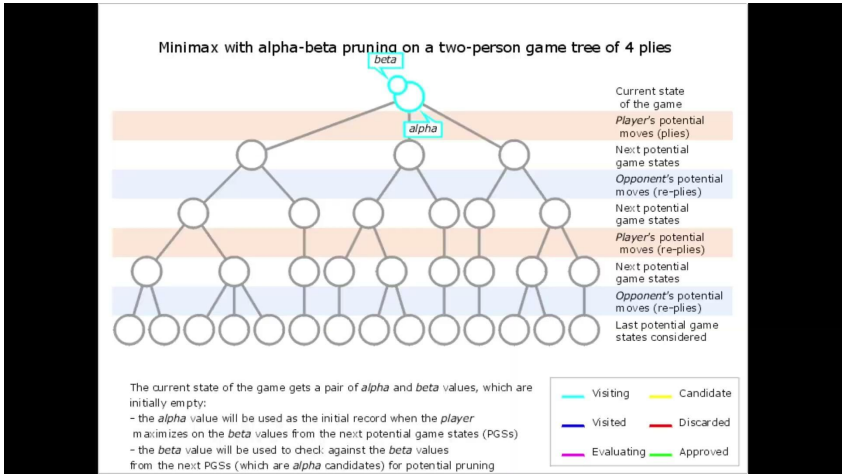
Stockfish



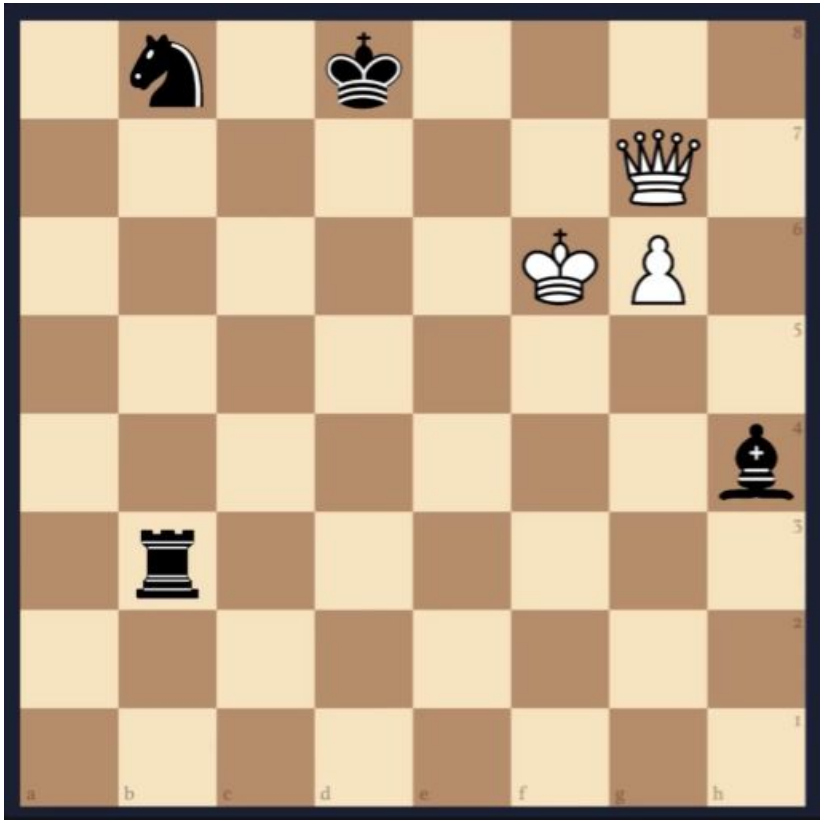
Depth: 1
Eval: +1.15

Game-Tree: Graph representing all possible positions

Heuristic: Alpha-Beta pruning + NNUE



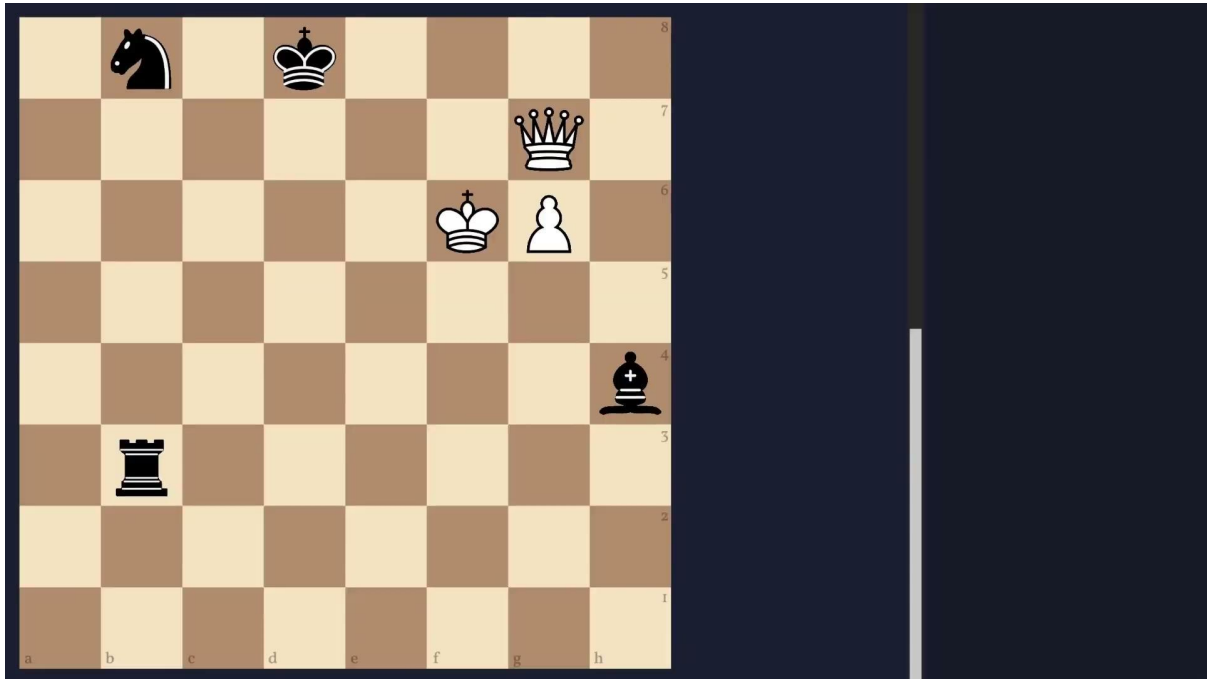
Computational Chess



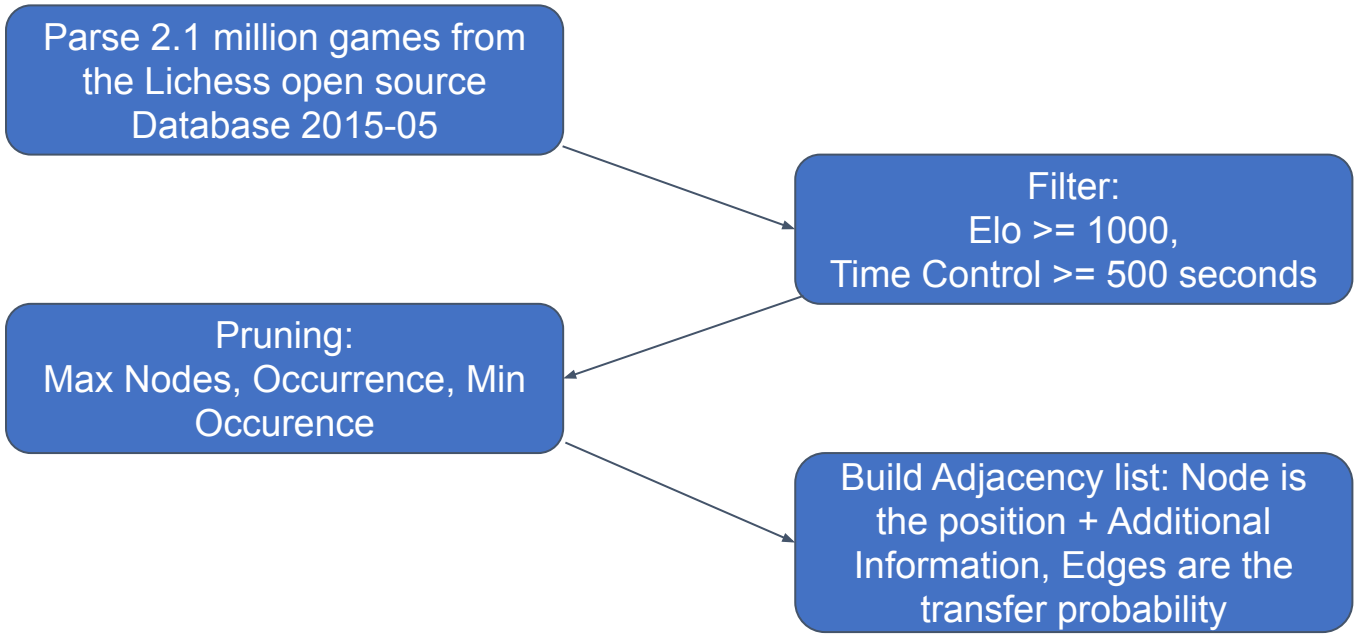
Computational Chess vs. Human Chess

White to move and mate in 549.

Computational approaches are not always helpful to humans

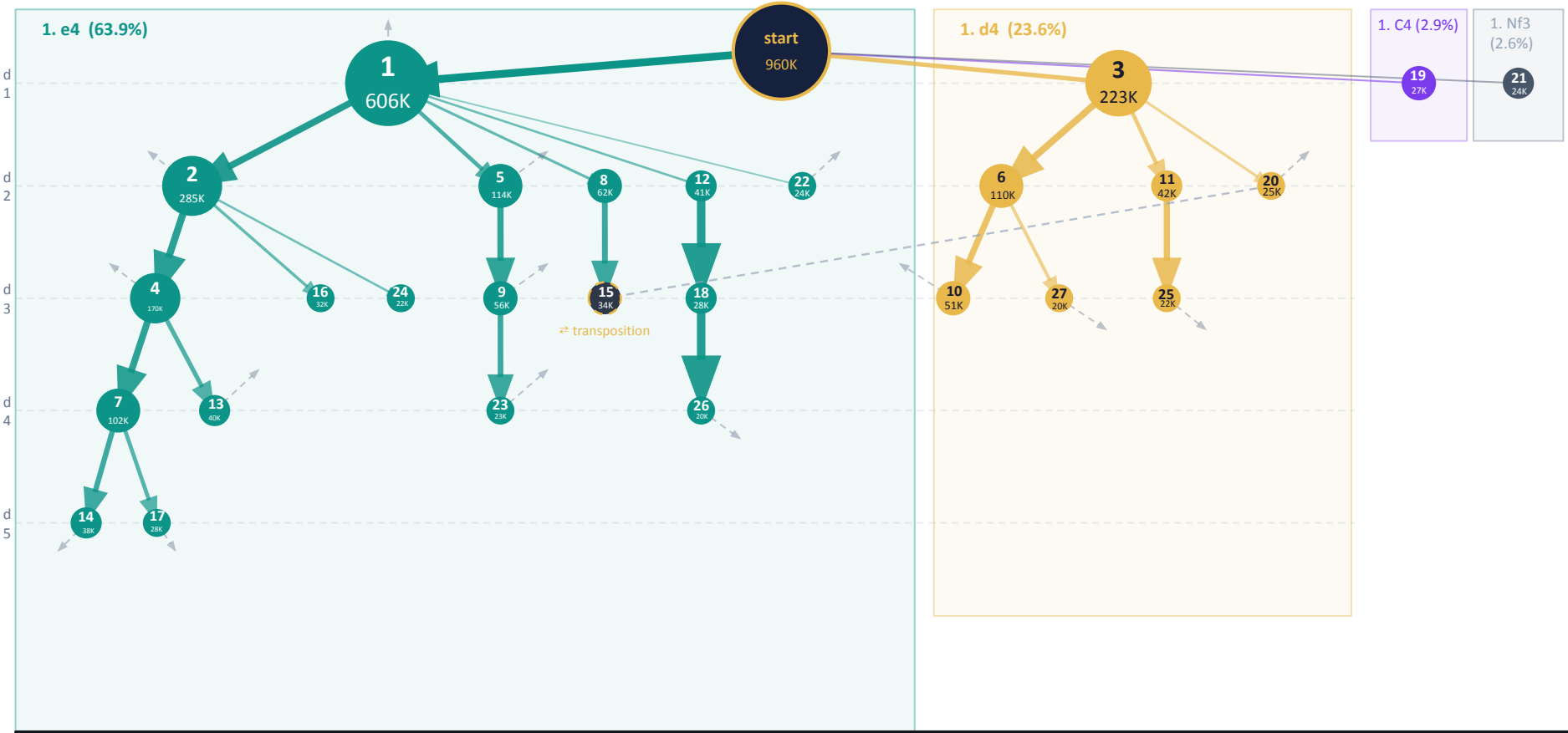


Building the Dataset



Chess Opening Graph — Branch Structure

Top 28 nodes · node size = visits · edge width = transition probability



Network exploration

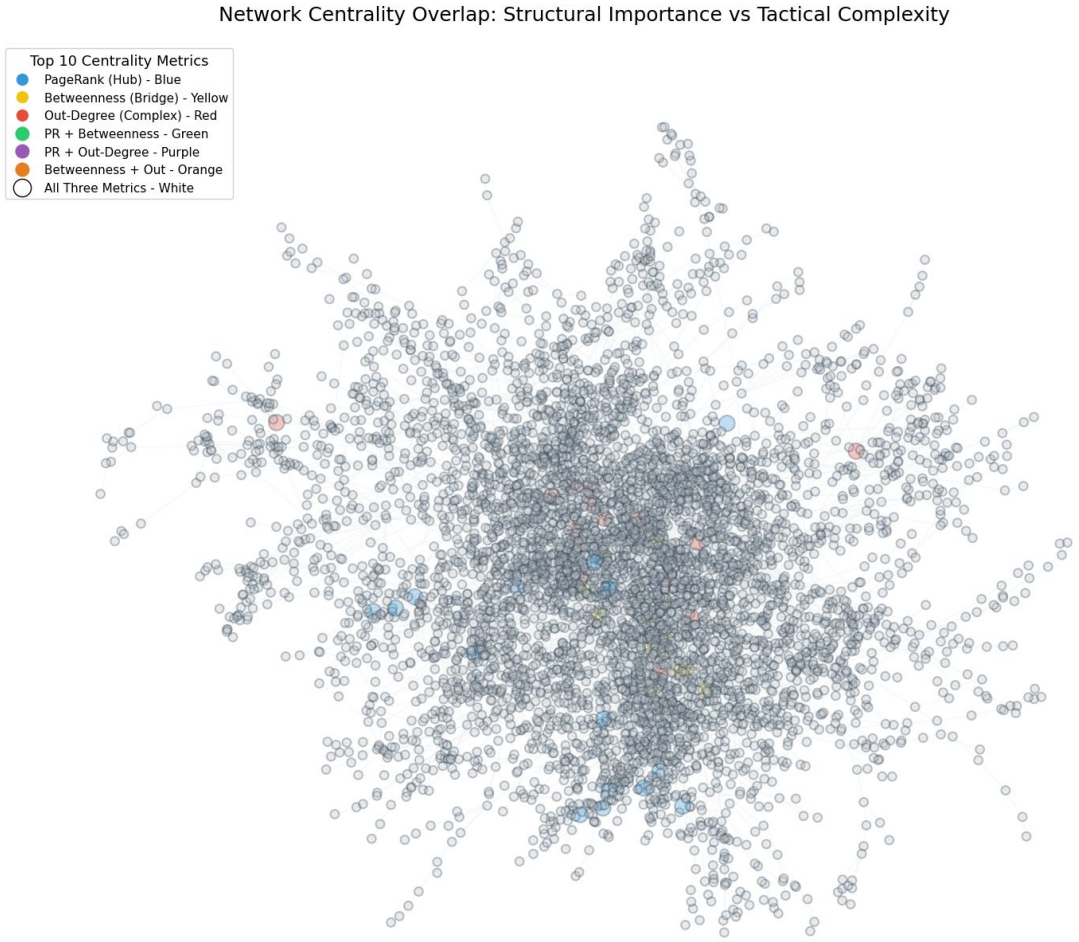
Distribution: 50.04% white wins, 3.9% draws, 45.6% black wins

Network density: 0.00023 (sparse)

6196 nodes, 9013 edges

Average out degree: 1.45
Max out degree: 15

Transitivity 0.000000

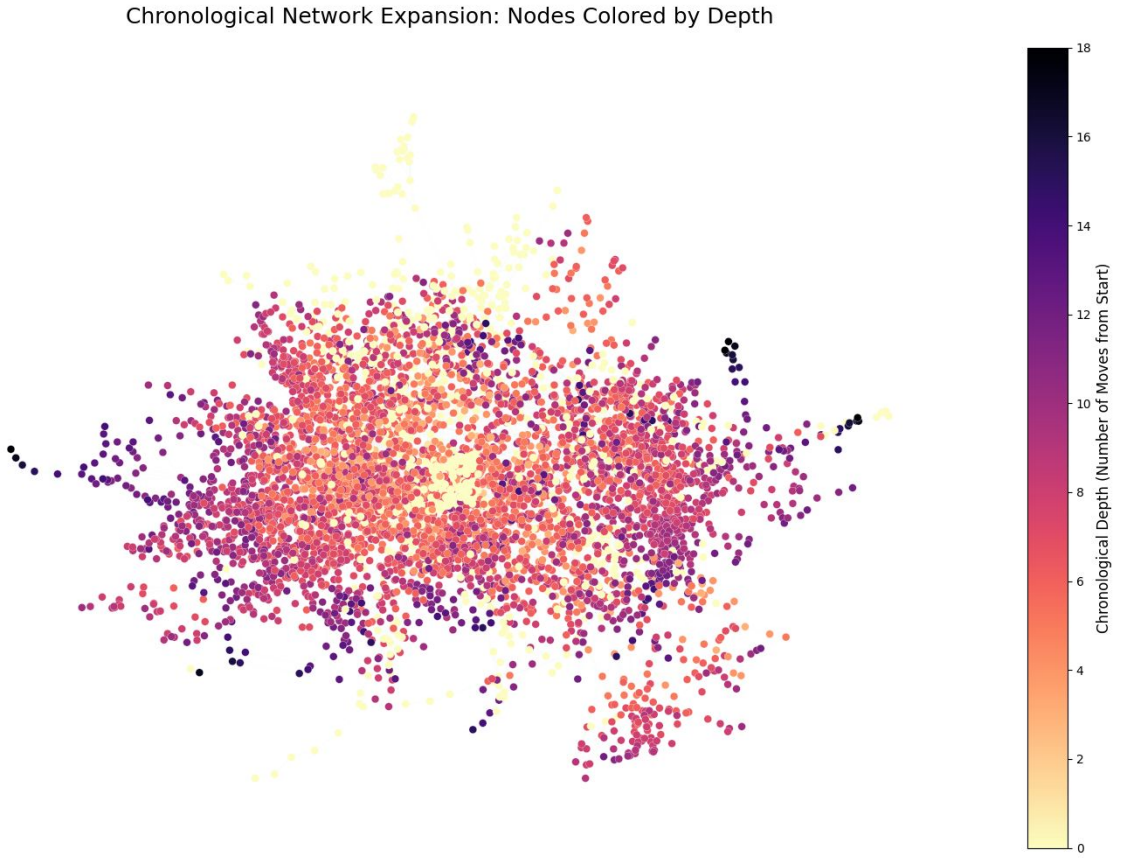


subtree-based clustering

Modularity 0.5244

Coverage 0.9293

Performance 0.5997



Our approach: a custom DFS + beam search

$$EV = \frac{W - L}{W + L + D}$$

Heuristic based on EV

DFS algorithm recursiveness

Greedy optimization based on beam search

Algorithm: Expectimax (DDFS + Beam search)

Input Node (Current state), path (Current path), score (Current score), Depth, MaxDepth, W (beam width), perspective, K (number of paths)

Result Recursion until finding the optimal path that maximizes the expected value

If Depth = MaxDepth **or** Node.outDegree == 0:

| **append** (path, score) **to** valid paths

ActivePlayer = Extract the current FEN structure

If ActivePlayer == "w":

Multiplier = 1

Else

Multiplier = -1

Candidates= ***new List***

Foreach nextNode **in** Node.succesors:

edgeProb = edge (Node, nextNode). probability

childEV = nextNode expected value

step = edgeProb * (childEV * multiplier)

append (nextNode, step) to Candidates

sort Candidates **key** descending **by** step

topCandidates = *keep only the W best candidates*

foreach (nextNode, step) **in** topCandidates:

RecursiveCall (nextNode, path + nextNode, score + step, depth + 1)

RecursiveCall (startNode, [startNode], 0.0, 0)

If perspective **is** white:

sort paths **key** desncending **by** EV of final node

elif perspective **is** black

sort paths **key** ascending **by** EV of final node

else

sort path **key** descending **by** accumulated path score

Return top K Paths.

The 6 Algorithms

Random Walk

FLOOR

Uninformed

✓ Returns ranked paths

Picks next node uniformly at random. Scores path with p×EV. Pure chance — no guidance.

Runtime

$O(d) = O(1)$

NX DFS

BASELINE

Uninformed

✗ No ranked paths

Standard NetworkX depth-first traversal. No scoring. Visits all reachable nodes up to depth 4.

Runtime

$O(V + E)$

NX BFS

BASELINE

Uninformed

✗ No ranked paths

Breadth-first traversal. Explores level by level. No scoring. Same quality limitation as NX DFS.

Runtime

$O(V + E)$

Dijkstra

OPTIMAL

Exact

✓ Returns ranked paths

Cost-inverted shortest path: cost=1/(p×EV). Runs once per boundary node at depth 4. Quality ceiling.

Runtime

$O(B \times (V+E) \log V)$

Expectimax

OURS

Heuristic

✓ Returns ranked paths

Recursive DFS scoring each successor by p(u→v)×EV(v). Beam width b=2, depth d=4. Returns top-k ranked paths.

Runtime

$O(b^d) = O(16)$

Beam Search

OPTIMAL

Heuristic

✓ Returns ranked paths

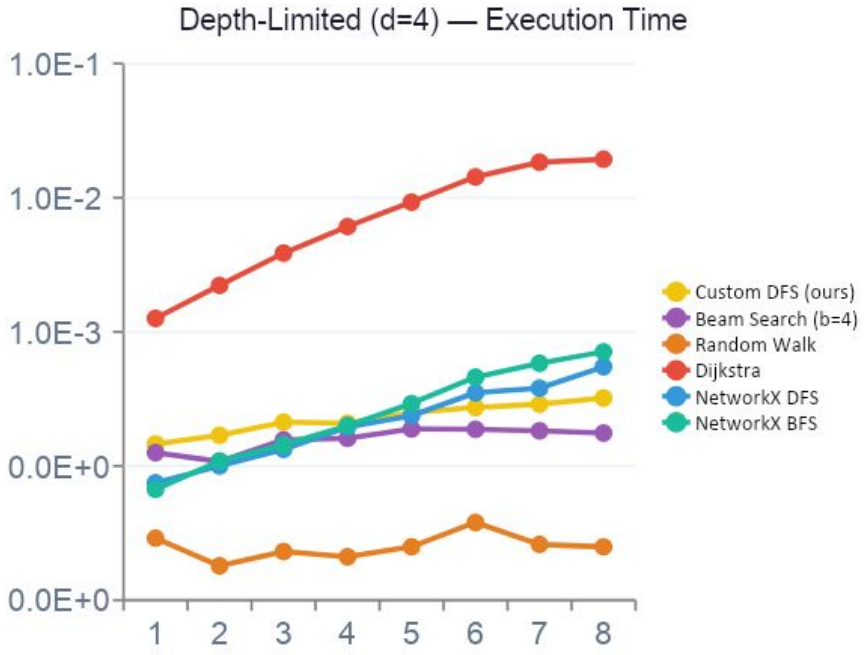
Global beam search b=4. Keeps top-4 candidates across entire frontier at each depth level.

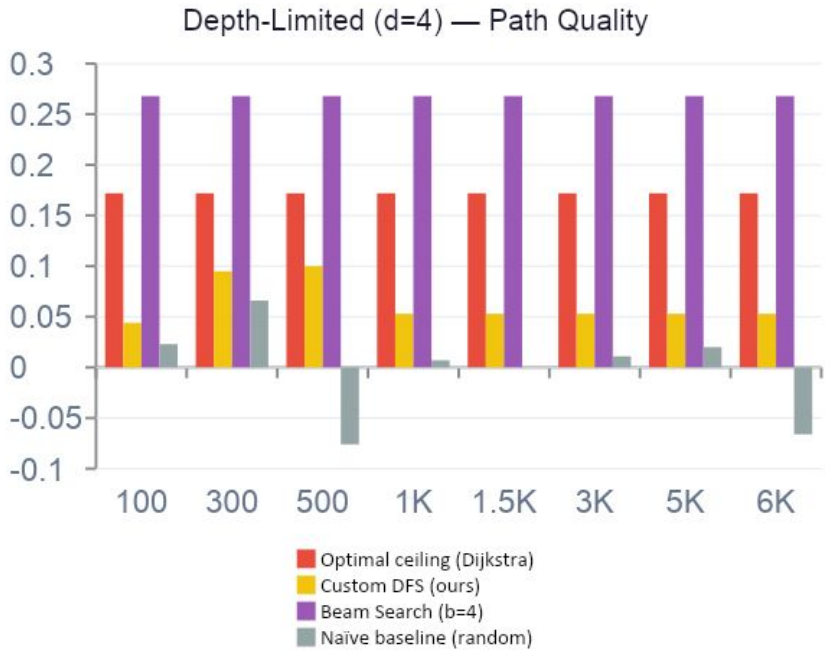
Runtime

$O(b \times N \times d)$

NX DFS and NX BFS excluded from quality comparisons — they return no ranked paths ·

Depth-Limited (d=4)





Speed — Random Walk fastest, Dijkstra slowest

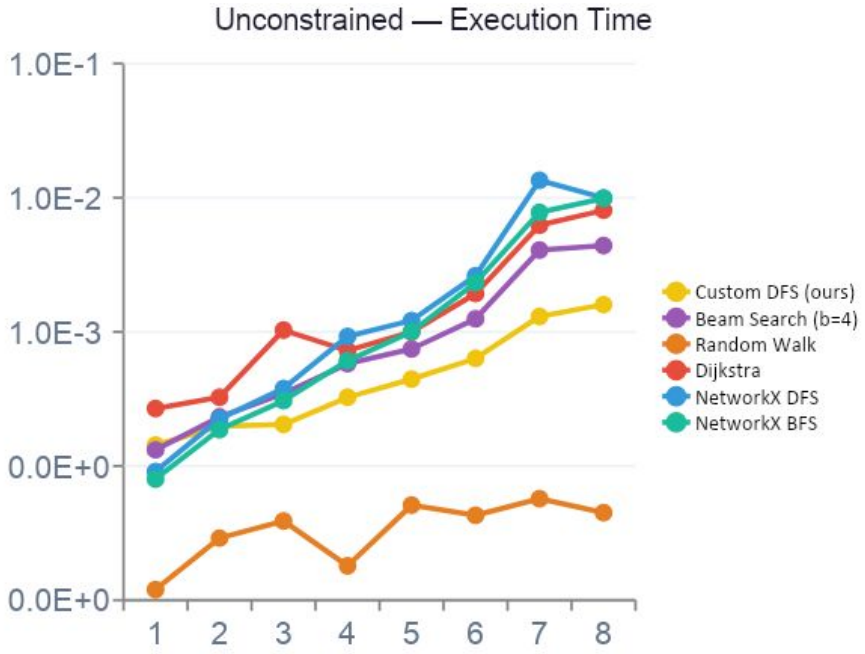
Random: ~0.02ms · Beam: 0.18ms · Custom DFS: 0.32ms · NX DFS/BFS: ~0.7ms · Dijkstra: 19.3ms.

Quality — Beam search is the highest performer

Random goes negative at 300, 500, 6K nodes. Beam Search tops at 0.268.Expectimax ends at 0.053 for graphs between 1 and 6k nodes

Verdict: Faster, but not better than Dijkstra — Beam search is the best performer

Unconstrained condition



Speed — Our algorithm is now the more efficient

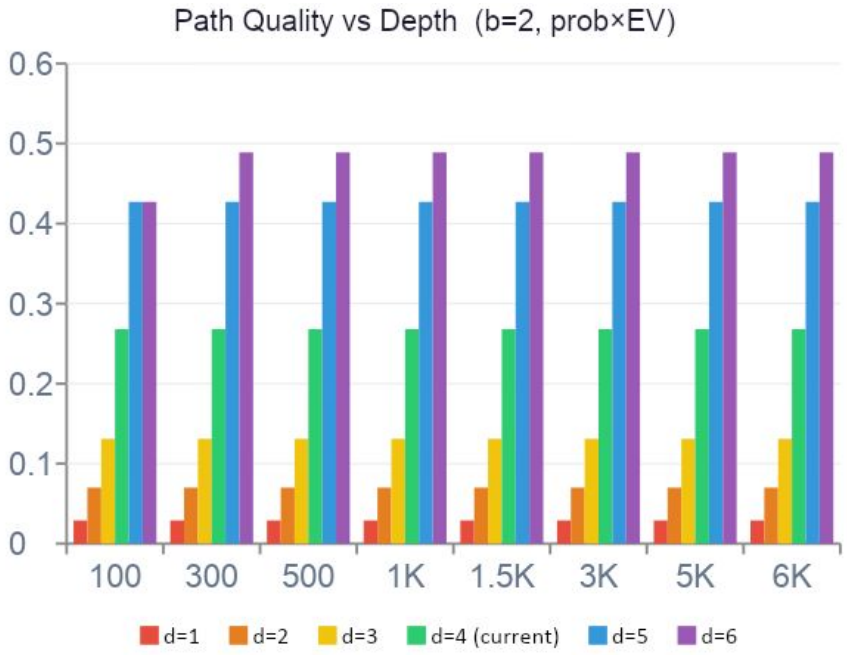
Without depth cap, the gap is reduced. RW is still the fastest, followed by

Quality — The volume of the graph plays in our favor

The bigger the graph, the better our algorithm performs. Although it doesn't beat Dijkstra. Beam search falls to 0.

Verdict: For larger graphs, our graph is a cheaper alternative than Dijkstra, with an acceptable loss of Path Quality

Ablation study – Depth ablation



Quality doubles at each depth increment

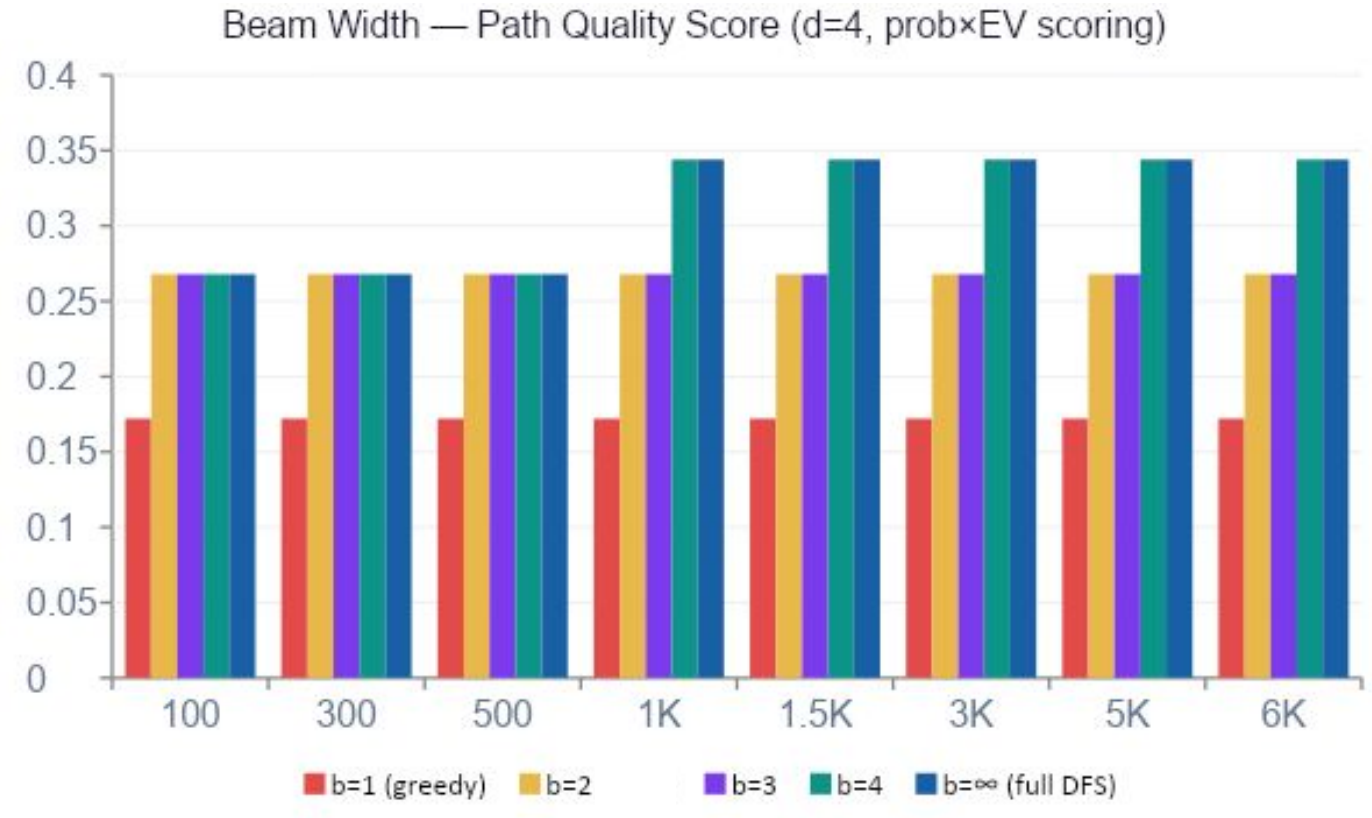
d=1: 0.029 → d=2: 0.070 → d=3: 0.131 → d=4: 0.268 → d=5: 0.427 → d=6: 0.489

D5 is the optimal setting

d=5 scores 0.427 vs d=4's 0.268. Runtime stays under 1ms. d=5 is the better operating depth — same cost, significantly better paths.

Verdict: Quality scales cleanly with depth. **d=5 is a free +59% upgrade over d=4** · d=6 adds +15% more at 1.5ms — still tractable.

Ablation Study — Beam ablation

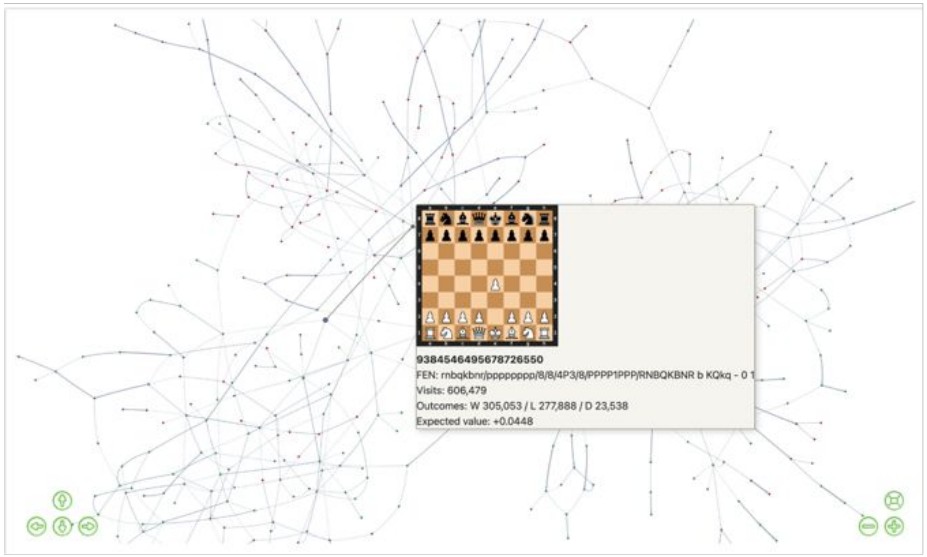


Findings: b=2 lossless up to b=3 · **b=4 = +29% quality** · scoring function ablation needs normalisation

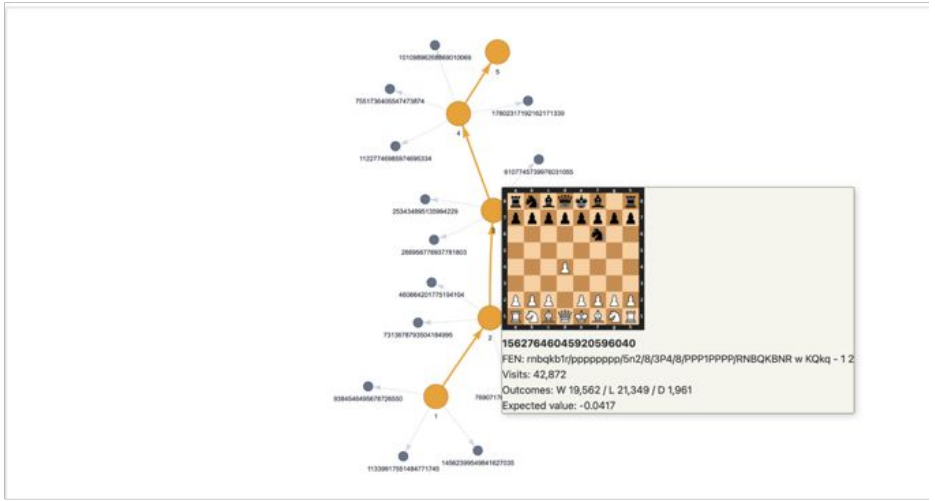
Graph Visualization

From the full opening tree to an inspected continuation with board state, visits, outcomes, and EV.

Global map: sparse, branching opening landscape



Focused path: ranked line becomes explainable



Node = position; edge = move probability

Demo focus: click a node, inspect the board, then follow the highest-ranked continuation.

Barnes-Hut force model

Top Ranked Opening Paths

https://lichess.org/study/PSQVkV81

#1 Semi-Benoni

1. d4 c5 2. d5 e5 3. e4 d6

Score: 0.1397

Ply	Visits	Move	
0	960,084	Starting position	
1	223,606	1. d4	
2	6,815	1...c5	gambit (3%)
3	2,424	2. d5	White advances
4	387	2...e5	
5	128	3. e4	
6	102	3...d6	Closed structure

E[v] -0.1373

#2 Indian Defense → Trompowsky

1. d4 Nf6 2. Bg5 g6 3. Nf3 Bg7

Score: 0.1288

Ply	Visits	Move	
0	960,084	Starting position	
1	223,606	1. d4	
2	42,872	1...Nf6	Indian Defense
3	1,787	2. Bg5	Trompowsky
4	184	2...g6	
5	277	3. Nf3	
6	249	3...Bg7	Fianchetto

E[v] -0.1205

#3 Sicilian Alapin

1. e4 c5 2. c3 d5 3. e5 Nc6

Score: 0.1259

Ply	Visits	Move	
0	960,084	Starting position	
1	606,479	1. e4	
2	114,913	1...c5	Sicilian
3	5,897	2. c3	Alapin
4	823	2...d5	strikes center
5	344	3. e5	
6	168	3...Nc6	tempo

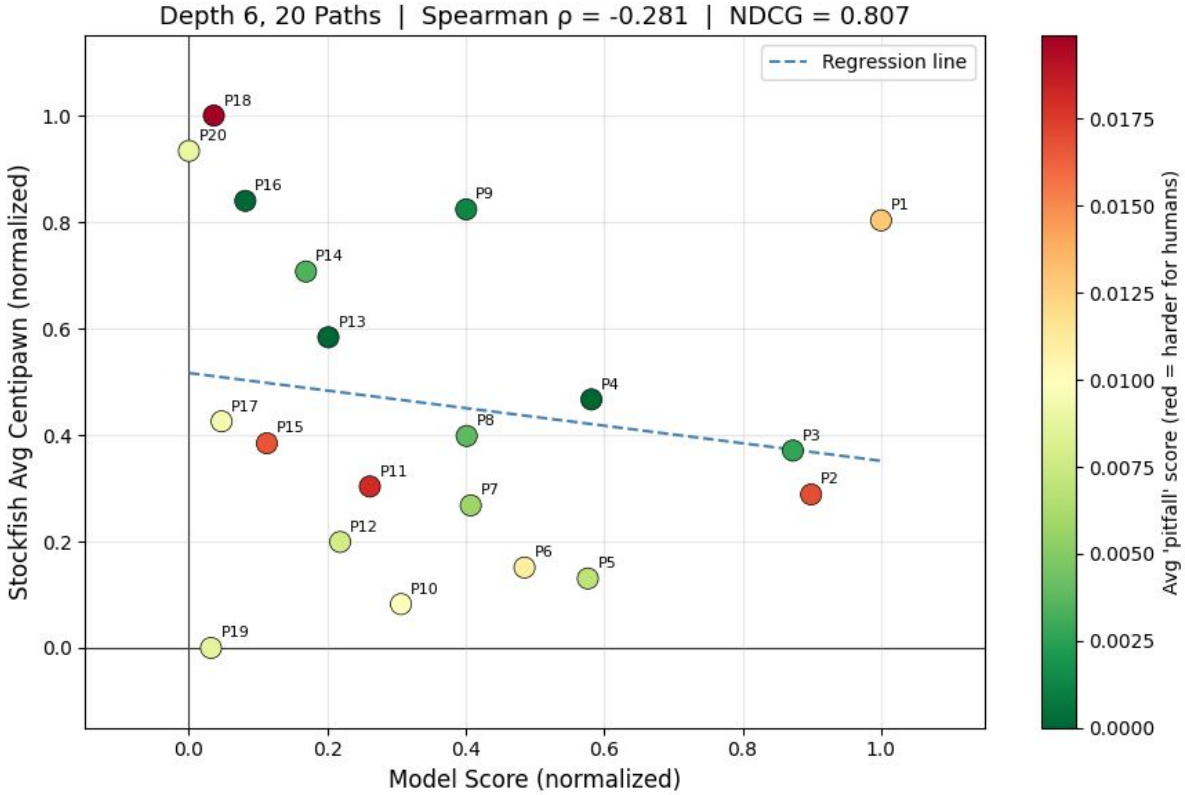
E[v] -0.2143

Stockfish comparison

Our “Ground Truth” comparison attempt

We do not want to replicate, but get an idea of overlap

Danger Score: Indicator for potential traps



LIVE DEMO

Demo

We will start from a .pgn.zst data file from lichess database and build the graph and then apply the needed analysis and visualizations.

Conclusion

Representing Chess as a graph does make sense, however more complex than we initially anticipated

Interesting findings:

- Chess results are quite interesting. Our algorithm likes unconventional (especially for black), but not “crazy” openings, e.g. Kings Gambit.
- First step towards providing hard evidence, that some lines are worth more taking and studying, even if those are not top engine recommendations.

Future steps:

- An in depth ablation study for our implementation and the ML implementation
- Try the models with a bigger sample that also allows beginner moves
- Feed the model with a more balanced dataset (W,L,D distribution amongst white and black pieces)